

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"Белгородский государственный технологический университет им. В. Г.
Шухова"
(БГТУ им. В.Г. Шухова)

Институт энергетики, информационных технологий и управляющих систем

Кафедра программного обеспечения вычислительной техники
и автоматизированных систем

Лабораторная работа № 2.1
по дисциплине: Дискретная математика
по теме: Алгоритмы порождения комбинаторных объектов

Выполнил: студент группы ПВ-221
Дорошенко Владимир Юрьевич
Проверил: ст. преподаватель
Бондаренко Т.В.

Белгород 2023

Цель работы: изучить основные комбинаторные объекты, алгоритмы их порождения, программно реализовать и оценить временную сложность алгоритмов.

Задания

1. Реализовать алгоритм порождения подмножеств.
2. Построить график зависимости количества всех подмножеств от мощности множества.
3. Построить графики зависимости времени выполнения алгоритмов п.1 на вашей ЭВМ от мощности множества.
4. Определить максимальную мощность множества, для которого можно получить все подмножества не более чем за час, сутки, месяц, год на вашей ЭВМ.
5. Определить максимальную мощность множества, для которого можно получить все подмножества не более чем за час, сутки, месяц, год на ЭВМ, в 10 и в 100 раз быстрее вашей.
6. Реализовать алгоритм порождения сочетаний.
7. Построить графики зависимости количества всех сочетаний из n по k от k при $n = (5, 7, 9)$.
8. Реализовать алгоритм порождения перестановок.
9. Построить график зависимости количества всех перестановок от мощности множества.
10. Построить графики зависимости времени выполнения алгоритма п.8 на вашей ЭВМ от мощности множества.
11. Определить максимальную мощность множества, для которого можно получить все перестановки не более чем за час, сутки, месяц, год на вашей ЭВМ.
12. Определить максимальную мощность множества, для которого можно получить все перестановки не более чем за час, сутки, месяц, год на ЭВМ, в 10 и в 100 раз быстрее вашей.
13. Реализовать алгоритм порождения размещений.
14. Построить графики зависимости количества всех размещений из n по k от k при $n = (5, 7, 9)$.

Задание 1

Реализовать алгоритм порождения подмножеств.

```
#include <iostream>
#include <vector>

using namespace std;

void getSubsets(vector<int> && u) {
    int size = u.size();
    int maxVector = 1 << size;
    for (int boolVector = 0; boolVector < maxVector; boolVector++) {
        cout << "{ ";
        for (int i = 0; i < size; i++) {
            if (boolVector >> i & 1) {
                cout << u[i] << " ";
            }
        }
        cout << "]\n";
    }
}

int main() {
    getSubsets({1, 2, 3});

    return 0;
}
```

Вывод :

```
{ }
{ 1 }
{ 2 }
{ 1 2 }
{ 3 }
{ 1 3 }
{ 2 3 }
{ 1 2 3 }
```

Задание 2

Построить график зависимости количества всех подмножеств

ОТ МОЩНОСТИ МНОЖЕСТВА.

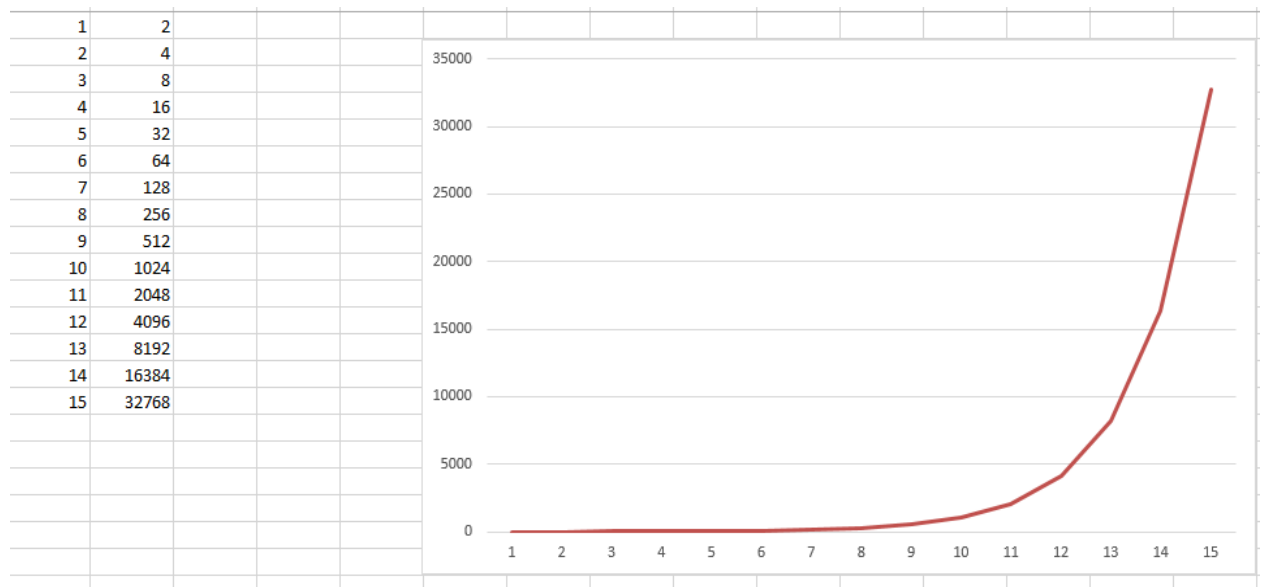
Код для вычисления данных для графика :

```
#include <iostream>
#include <vector>

int main() {
    vector<int> set;
    for (int i = 1; i <= 15; i++) {
        set.push_back(i);
        cout << i << ' ' << (1 << set.size()) << '\n';
    }

    return 0;
}
```

График :



Задание 3

Построить графики зависимости времени выполнения алгоритмов п.1 на вашей ЭВМ от мощности множества.

Код, считающий время выполнения :

```
#include <iostream>
#include <vector>

int main() {
    setvbuf(stdout, nullptr, _IOFBF, 512);
    ios_base::sync_with_stdio(false);

    vector<int32_t> set;
    vector<double> times;

    for (int i = 1; i <= 15; i++) {
        set.push_back(i);

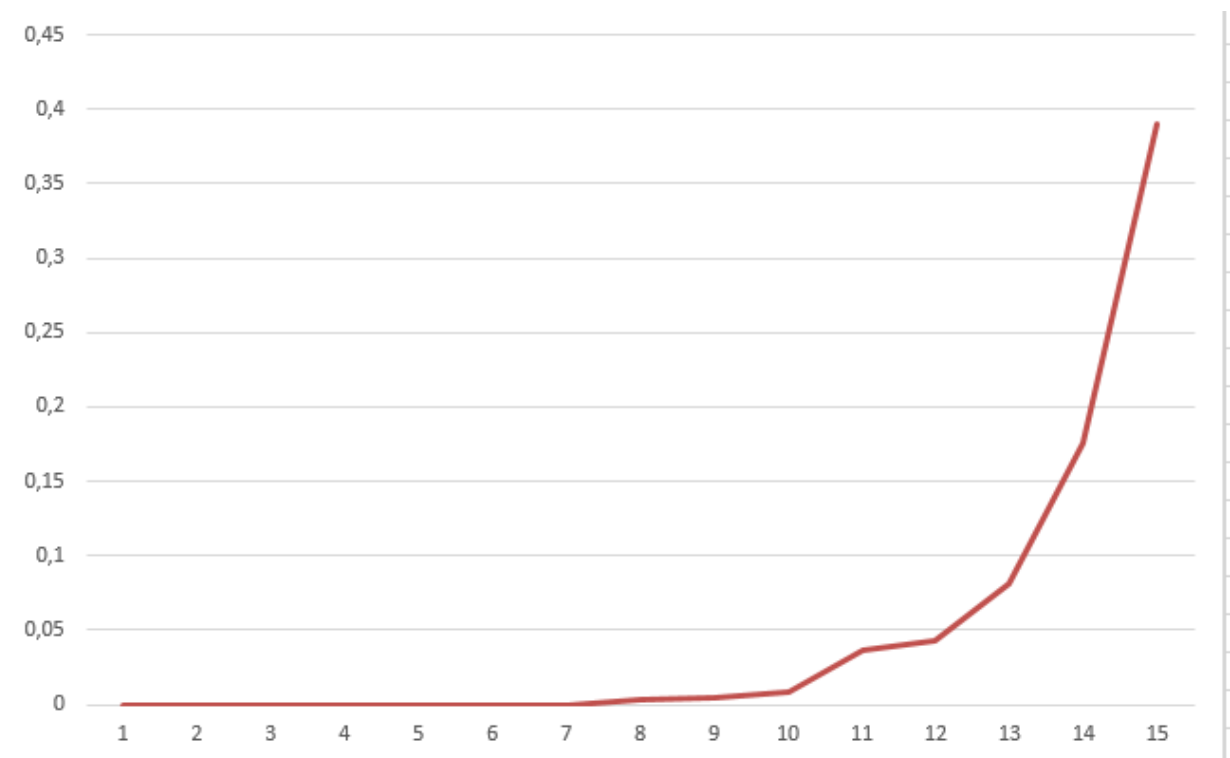
        time_t start = clock();
        getSubsets(set);
        time_t end = clock();

        times.push_back(static_cast<double>(end - start) / CLOCKS_PER_SEC);
    }

    for (int i = 1; i <= 15; i++) {
        cout << i << ' ' << times[i - 1] << '\n';
    }

    return 0;
}
```

График :



Задание 4

Определить максимальную мощность множества, для которого можно получить все подмножества не более чем за час, сутки, месяц, год на вашей ЭВМ.

Мощность множества	Время генерации всех подмножеств на моей ЭВМ
28	Час
32	Сутки
37	Месяц
41	Год

Задание 5

Определить максимальную мощность множества, для которого можно получить все подмножества не более чем за час, сутки, месяц, год на ЭВМ, в 10 и в 100 раз быстрее вашей.

Для ЭВМ в 10 раз мощнее

Мощность множества	Время генерации всех подмножеств на ЭВМ в 10 раз мощнее
31	Час
36	Сутки
41	Месяц
44	Год

Для ЭВМ в 100 раз мощнее

Мощность множества	Время генерации всех подмножеств на ЭВМ в 10 раз мощнее
34	Час
39	Сутки
44	Месяц
47	Год

Задание 6

Реализовать алгоритм порождения сочетаний.

```
#include <iostream>
#include <vector>

void getComb(const int n, const int k, const int i, const int b, vector<int>
&set) {
    for (int x = b; x <= n - k + i; x++) {
        set.push_back(x);
        if (i == k) {
            cout << "{ ";
            for (auto &item : set) {
                cout << item << " ";
            }
            cout << "}\n";
        } else {
            getComb(n, k, i + 1, x + 1, set);
        }

        set.pop_back();
    }
}

void getComb(const int n, const int k) {
    if (k == 0) {
        cout << "{ } \n";
        return;
    }

    vector<int> tmp;
    getComb(n, k, 1, 1, tmp);
}

int main() {
    setvbuf(stdout, nullptr, _IOFBF, 512);
    ios_base::sync_with_stdio(false);

    for (int i = 0; i <= 4; i++) {
        getComb(4, i);
    }

    return 0;
}
```

Вывод :

```
{ }
{ 1 }
{ 2 }
{ 3 }
{ 4 }
{ 1 2 }
{ 1 3 }
{ 1 4 }
{ 2 3 }
{ 2 4 }
{ 3 4 }
{ 1 2 3 }
{ 1 2 4 }
{ 1 3 4 }
{ 2 3 4 }
{ 1 2 3 4 }
```

Задание 7

Построить графики зависимости количества всех сочетаний из n по k от k при $n = (5, 7, 9)$.

k	C_n^k при $n = 5$	C_n^k при $n = 7$	C_n^k при $n = 9$
0	1	1	1
1	5	7	9
2	10	21	36
3	10	35	84
4	5	35	126
5	1	21	126
6	-	7	84
7	-	1	36
8	-	-	9
9	-	-	1

График при $n = 5$:

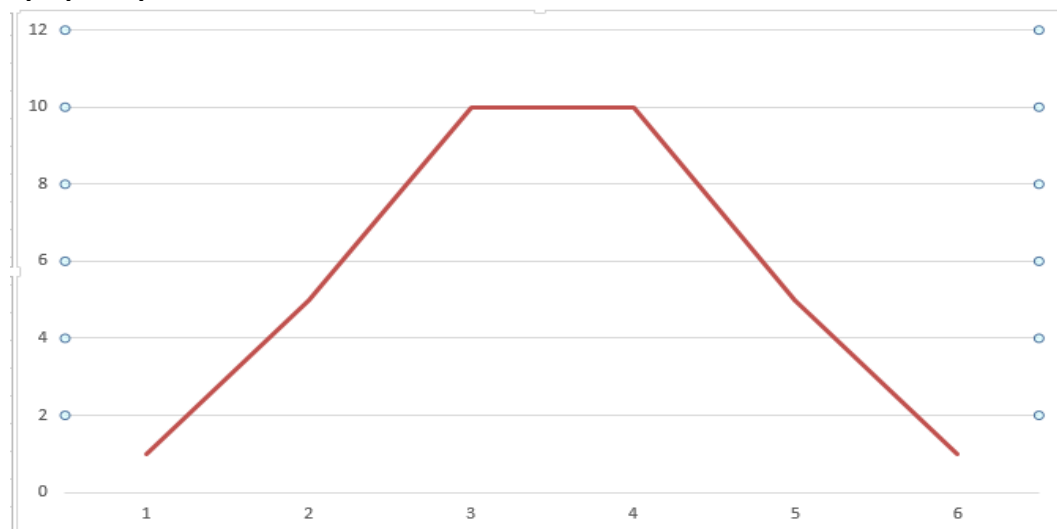


График при $n = 7$:

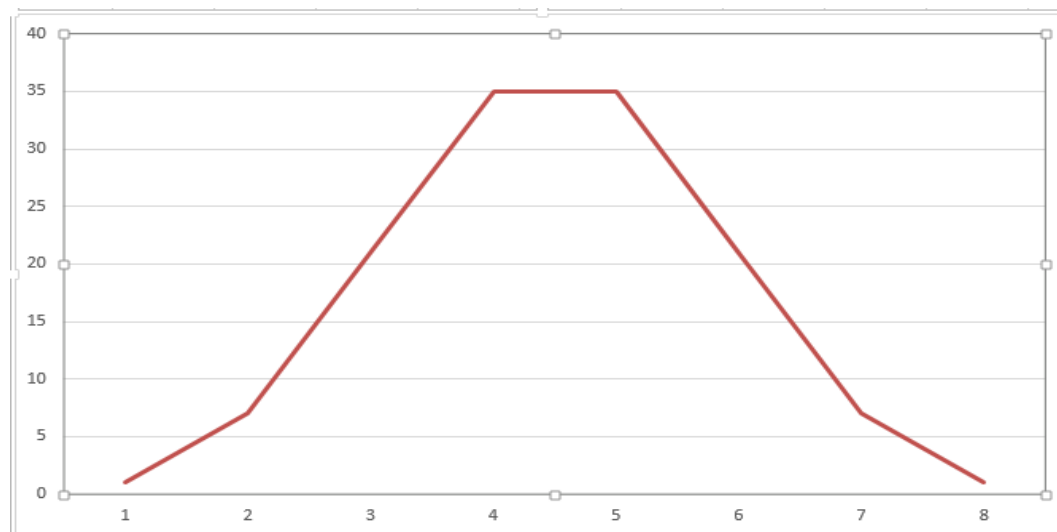
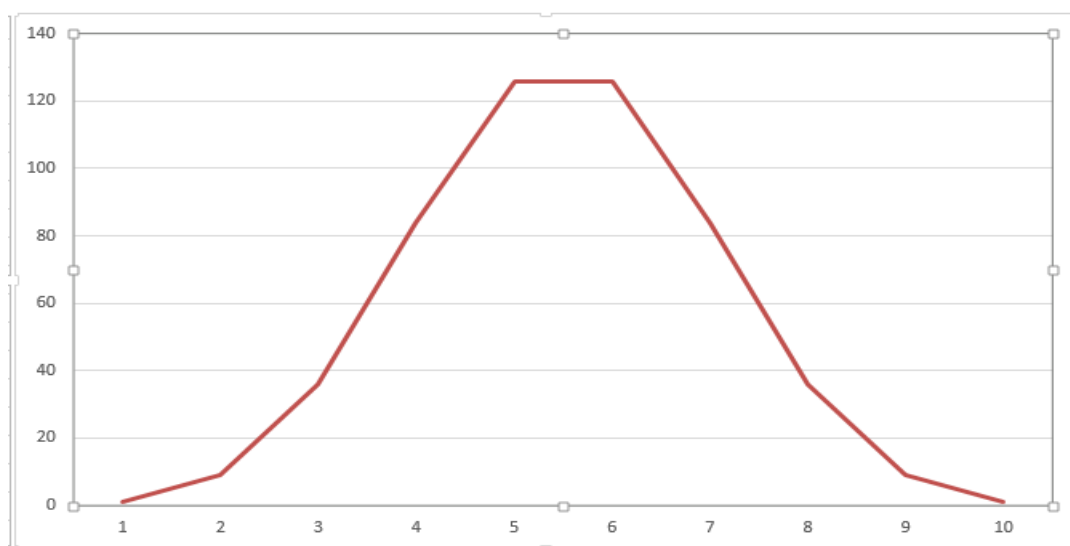


График при $n = 9$:



Задание 8

Реализовать алгоритм порождения перестановок.

```
#include <iostream>
#include <vector>
#include <set>

set<int> operator-(set<int> set, const int x) {
    set.erase(x);

    return set;
}

void getPermutations(const set<int> u, const int n, const int i,
                    vector<int> &p) {
    for (auto x = u.begin(); x != u.end(); x++) {
        p[i - 1] = *x;
        if (i == n) {
            cout << "{ ";
            for (auto &item : p) {
                cout << item << ' ';
            }
            cout << "}\n";
        } else {
            getPermutations(u - *x, n, i + 1, p);
        }
    }
}

void getPermutations(const set<int> &u) {
    if (u.empty()) {
        cout << "{ }";
        return;
    }

    vector<int> tmp(u.size());
    getPermutations(u, u.size(), 1, tmp);
}

int main() {
    setvbuf(stdout, nullptr, _IOFBF, 512);
    ios_base::sync_with_stdio(false);

    set<int> set;

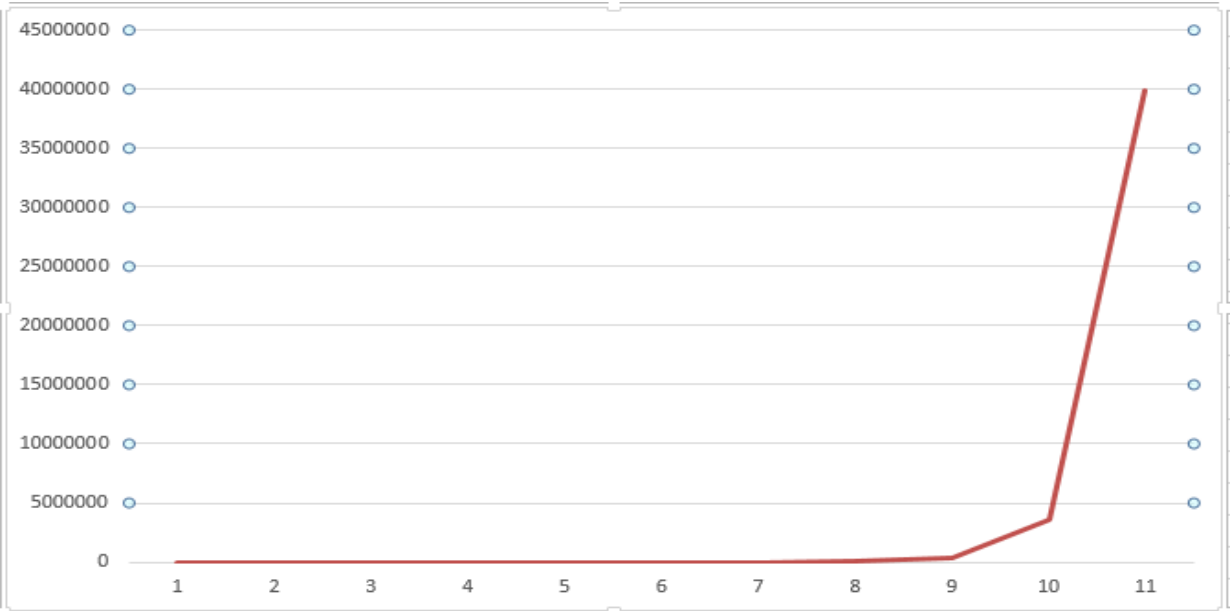
    for (int i = 1; i <= 4; i++) {
        set.insert(i);
        getPermutations(set);
    }

    return 0;
}
```

Задание 9

Построить график зависимости количества всех перестановок от мощности множества.

Мощность множества	Количество перестановок
1	1
2	2
3	6
4	24
5	120
6	720
7	5040
8	40320
9	362880
10	3628800
11	39916800



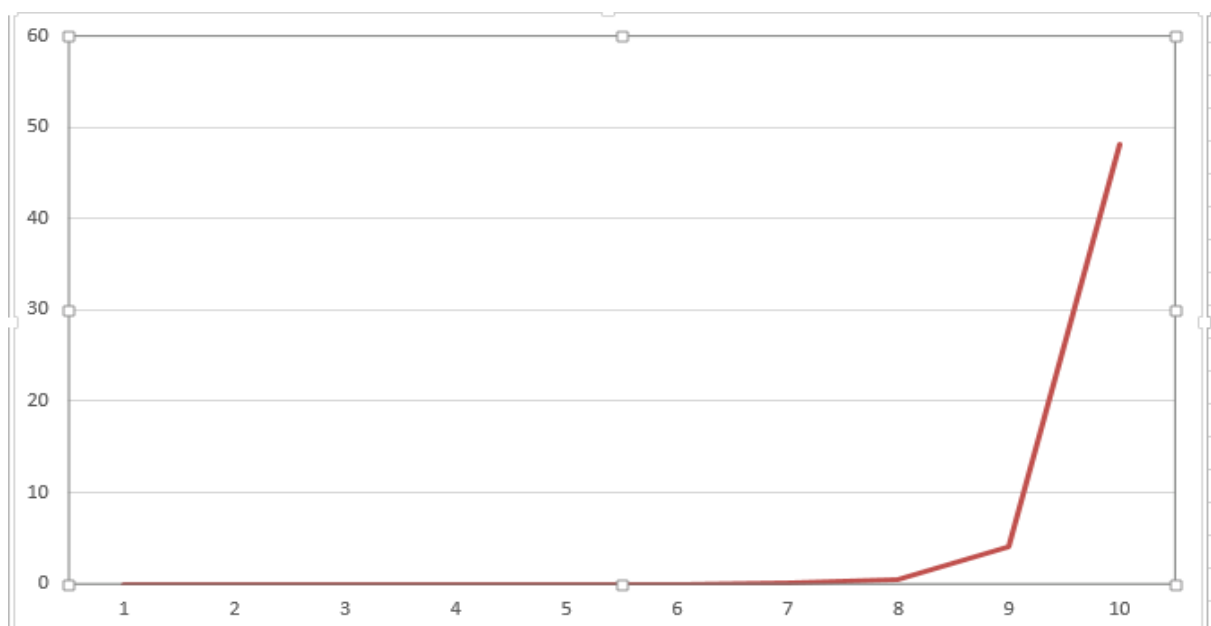
Задание 10

Построить графики зависимости времени выполнения алгоритма п.8 на вашей ЭВМ от мощности множества.

Код для получения времени выполнения моей ЭВМ такой же как в Задании 3;

Мощность множества	Количество перестановок
1	0
2	0
3	0
4	0
5	0.001
6	0.009
7	0.096
8	0.439
9	4.107
10	48.188

График :



Задание 11

Определить максимальную мощность множества, для которого можно получить все перестановки не более чем за час, сутки, месяц, год на вашей ЭВМ.

Мощность множества	Время генерации всех подмножеств на ЭВМ в 10 раз мощнее
11	Час
13	Сутки
14	Месяц
15	Год

Задание 12

Определить максимальную мощность множества, для которого можно получить все перестановки не более чем за час, сутки, месяц, год на ЭВМ, в 10 и в 100 раз быстрее вашей.

Для ЭВМ в 10 раз мощнее

Мощность множества	Время генерации всех подмножеств на ЭВМ в 10 раз мощнее
12	Час
14	Сутки
15	Месяц
16	Год

Для ЭВМ в 100 раз мощнее

Мощность множества	Время генерации всех подмножеств на ЭВМ в 10 раз мощнее
13	Час
15	Сутки
16	Месяц
17	Год

Задание 13

Реализовать алгоритм порождения размещений.

```
#include <iostream>
#include <vector>
#include <set>

void getAccommodations(const set<int> &u, const int n, const int k, const int
i, vector<int> &p) {
    for (auto x = u.begin(); x != u.end(); x++) {
        p[i - 1] = *x;
        if (i == k) {
            cout << "{ ";
            for (auto &item: p) {
                cout << item << " ";
            }
            cout << "}\n";
        } else {
            getAccommodations(u - *x, n, k, i + 1, p);
        }
    }
}

void getAccommodations(const set<int> &u, const int k) {
    if (u.empty() || k == 0) {
        cout << "{ }";
        return;
    }

    vector<int> tmp(k);
    getAccommodations(u, u.size(), k, 1, tmp);
}

int main() {
    setvbuf(stdout, nullptr, _IOFBUF, 512);
    ios_base::sync_with_stdio(false);

    set<int> set{1, 2, 3, 4};

    for (int i = 1; i <= set.size(); i++) {
        getAccommodations(set, i);
    }

    return 0;
}
```

Задание 14

Построить графики зависимости количества всех размещений из n по k от k при $n = (5, 7, 9)$.

k	C_n^k при $n = 5$	C_n^k при $n = 7$	C_n^k при $n = 9$
0	1	1	1
1	5	7	9
2	20	42	72
3	60	210	504
4	120	840	3024
5	120	2520	15120
6	-	5040	60480
7	-	5040	181440
8	-	-	362880
9	-	-	362880

График при $n = 5$:

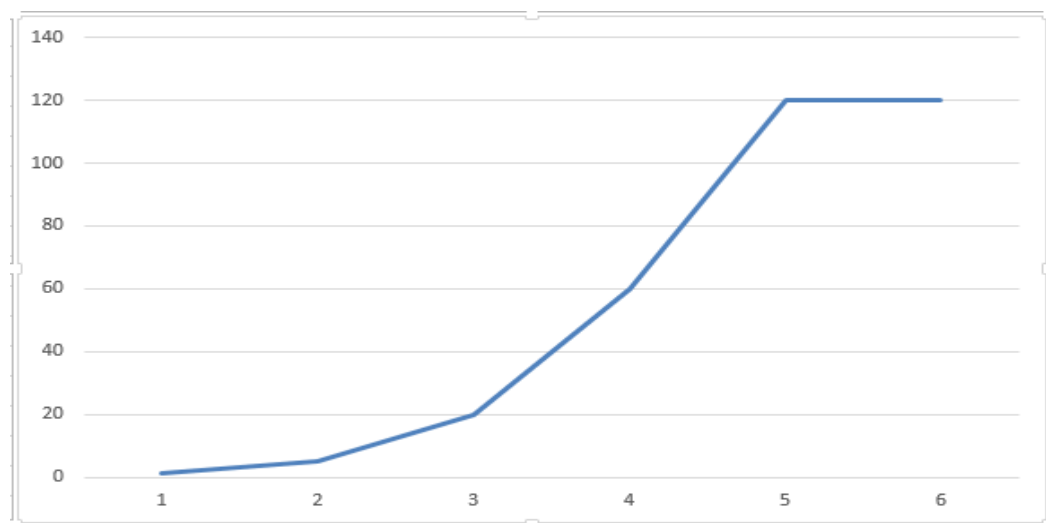


График при $n = 7$:

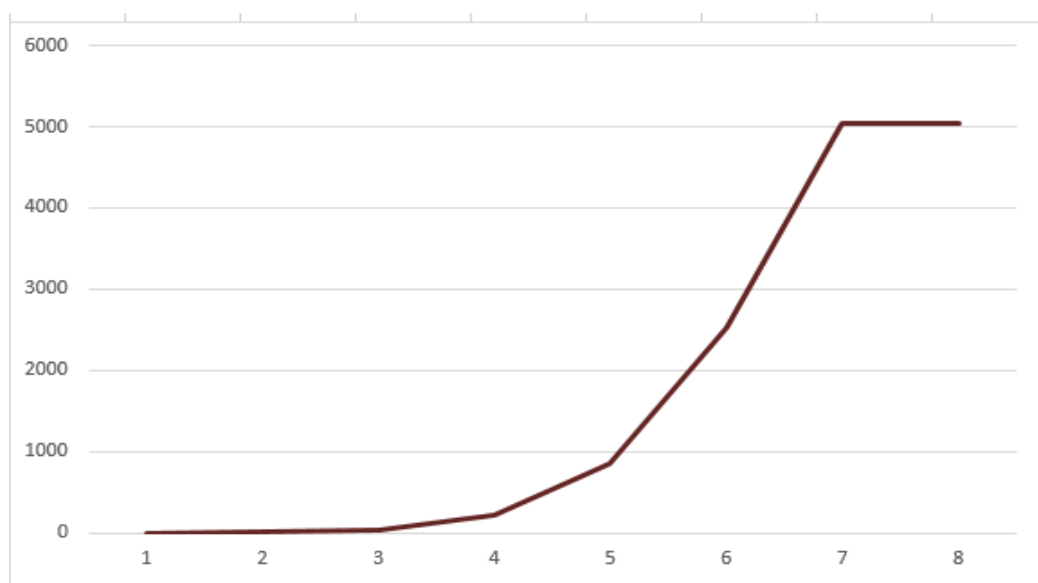
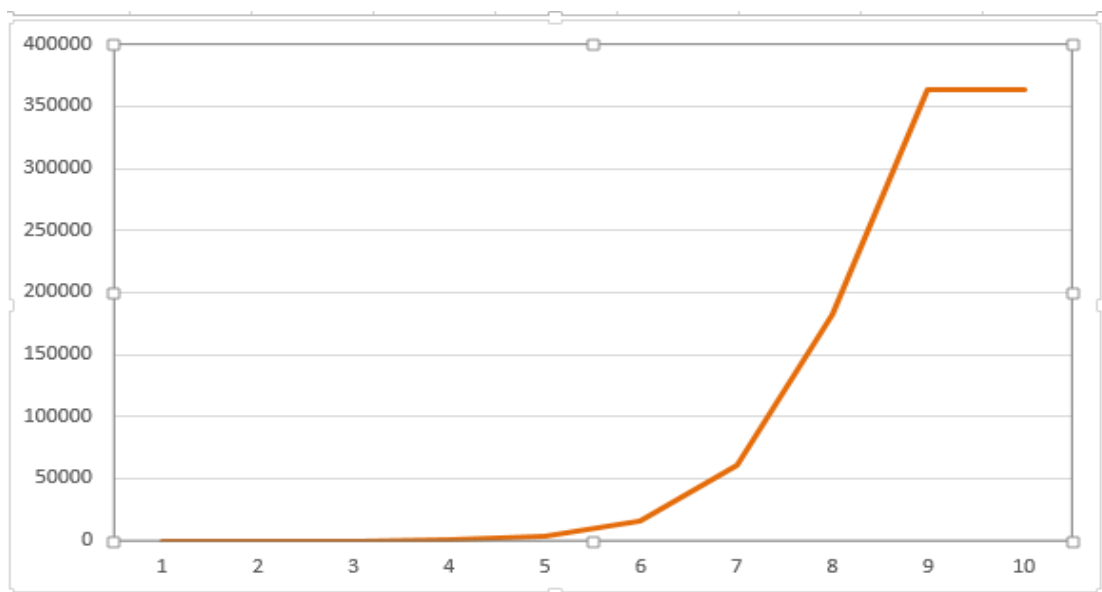


График при $n = 9$:



Вывод : Я изучил основные комбинаторные объекты, алгоритмы их порождения, программно реализовал и оценил временную сложность алгоритмов.